Trilha *RTL Synthesis* – Roteiro inicial de nivelamento - Git

Henrique do Nascimento Cunha, MSc.

RESIDÊNCIA EM MICROELETRÔNICA - POLO UFCG

Git

- O que é Git?
- Sistema de controle de versões
- Rastrear alterações em arquivos
- Coordenar o trabalho em equipe durante o desenvolvimento

Git

Por que usar Git?

Permite trabalhar em equipe sem conflitos.

Ou com formas de gerenciar conflitos

Mantém um histórico completo de alterações

Integra-se com plataformas como GitHub, GitLab e Bitbucket

Git

Como a idéia é usar Git colaborativamente, vamos nos dividir em duplas e criar um repositório compartilhado

Em qualquer ambiente de desenvolvimento, o trabalho em equipe é essencial. Assim vamos usar esse momento para exercitar nossa comunicação e conhecer os colegas

Também é esperado que os vocês se ajudem durante o processo

Git

Dividam-se em duplas e escolham entre vocês quem será aluno A e aluno B

O roteiro adiante precisará dessa divisão

Git

PARTE 1 – Configuração Inicial (os dois alunos de cada grupo fazem em seu PC)

Verificar se o git está instalado:

```
henrique:~$ git --version
git version 2.47.3
henrique:~$
```

Git

PARTE 1 – Configuração Inicial (os dois alunos de cada grupo fazem em seu PC)

Configurar nome e email:

```
henrique:~$ git config --global user.name "Henrique Cunha"
henrique:~$ git config --global user.email "henrique.cunha@ifpb.edu.br"
henrique:~$
```

Git

PARTE 2 – Criar o Repositório no GitHub (aluno A da dupla faz)

- Acesse github.com e faça login.
- Clique no ícone **+** (canto superior direito) → **New repository**.
- Preencha:
- **Repository name:** `roteiro-nivelamento`
- **Description:** "Repositório para atividade colaborativa de novelamento do CI-Expert"
- Deixe como **público** e adicione o colega como colaborador
- **Não** marque "Initialize this repository with a README". (Vamos criar depois)
- Clique em **Create repository**.
- Na tela seguinte, copie a URL do repositório (formato: `https://github.com/usuario/roteiro-nivelamento.git`).

Git

PARTE 3 – Adicionar o Colega como Colaborador (quem criou o repositório faz)

- No repositório recém-criado, vá em Settings → Collaborators.
- Clique em Add people e adicione o username ou e-mail do colega.
- O colega receberá um convite por e-mail e precisará aceitar.

Git

PARTE 4 – Criar Repositório Local e Vincular ao GitHub (ambos fazem, mas um de cada vez)

- Aluno que criou o repositório no GitHub:
- Crie uma pasta para o projeto:

```
henrique:~$ mkdir roteiro_nivelamento
henrique:~$ cd roteiro_nivelamento/
henrique:~/roteiro_nivelamento$
```

Git

- Inicie um repositório git local:

```
henrique:~/roteiro_nivelamento$ git init
```

Git

- Saída esperada:

```
henrique:~/roteiro_nivelamento$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/henrique/roteiro_nivelamento/.git/
henrique:~/roteiro_nivelamento$
```

Git

Aluno A crie um arquivo inicial

```
henrique:~/roteiro_nivelamento$ echo "# Atividade em Dupla - Git e GitHub" > README.md
henrique:~/roteiro_nivelamento$ ls
README.md
henrique:~/roteiro_nivelamento$
```

Git

- Aluno A adicione e faça o primeiro commit

```
henrique:~/roteiro_nivelamento$ git add README.md
git commit -m "Primeiro commit: adiciona README"
[master (root-commit) 4165845] Primeiro commit: adiciona README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
henrique:~/roteiro_nivelamento$
```

Git

- Aluno A vincule ao repositório no github

```
henrique:~/roteiro_nivelamento$ git remote add origin https://github.com/henriq
uecunha2/roteiro-nivelamento.git
henrique:~/roteiro_nivelamento$
```

Git

- Aluno A envie para o github:

```
henrique:~/roteiro_nivelamento_ci_expert$ git push -u origin main
Username for 'https://github.com': henriquecunha2
Password for 'https://henriquecunha2@github.com':
remote: Permission to henriquecunha2/roteiro_nivelamento_ci_expert.git de
nied to henriquecunha2.
fatal: unable to access 'https://github.com/henriquecunha2/roteiro_nivela
mento_ci_expert/': The requested URL returned error: 403
```

Git

- Envie para o github:

Isso é um problema! O github desde 2021 só permite acesso por token.

Git

Aqui está o passo a passo para resolver:

Como gerar um Token de Acesso Pessoal no GitHub

- **Acesse as configurações do GitHub:**
- Clique na sua **foto de perfil** (canto superior direito)
- Vá em **Settings** → **Developer settings** (final da lista, à esquerda)
- **Crie um novo token:**
- Clique em **Personal access tokens** → **Tokens (classic)**
- Clique em **Generate new token** → **Generate new token (classic)**
- **Configure o token:**
- **Note:** "Acesso para atividade em dupla" (ou algo que lembre)
- **Expiration:** Escolha um prazo
- **Scopes (selecionar):**
- ☑ **repo** (acesso completo a repositórios)
- ☑ **workflow** (se for usar Actions depois)
- **Clique em "Generate token"**
- **IMPORTANTE:** Copie o token AGORA! (É a única vez que ele aparece)

Git

- Agora é só fazer o comando de novo e colocar o texto do tokne no lugar da senha:

```
henrique:~/roteiro_nivelamento_ci_expert$ git push -u origin main
Username for 'https://github.com': henriquecunha2
Password for 'https://henriquecunha2@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 266 bytes | 266.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/henriquecunha2/roteiro_nivelamento_ci_expert
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Git

- Agora o aluno B da dupla clona o repositório:

```
henrique:~/roteiro$ git clone https://github.com/henriquecunha2/roteiro_n
ivelamento_ci_expert
Cloning into 'roteiro_nivelamento_ci_expert'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Git

- Se quiser ver as operações realizadas até o momento:

```
henrique:~/roteiro_nivelamento_ci_expert$ git log
commit fbe3841c433b1e663459af5b7a88fbdef211f556 (HEAD -> main, origin/mai
n)
Author: henriquecunha2 <henrique.cunha@ifpb.edu.br>
Date:   Thu Mar 19 13:12:03 2026 -0400

    Primeiro commit: adiciona README
henrique:~/roteiro_nivelamento_ci_expert$
```

Git

- PARTE 5 – Fluxo Colaborativo Básico
- Aluno B, cria um arquivo e adiciona:

```
henrique:~/roteiro_nivelamento_ci_expert$ echo "Primeira contribuição do
Aluno B" > contribuicao-a.txt
henrique:~/roteiro_nivelamento_ci_expert$ git add contribuicao-a.txt
henrique:~/roteiro_nivelamento_ci_expert$ git commit -m "Adiciona arquivo
 do Aluno A"
```

Git

- Aluno B, faz commit e push:

```
henrique:~/roteiro_nivelamento_ci_expert$ git commit -m "Adiciona arquivo
 do Aluno A"
[main 0b7c1f0] Adiciona arquivo do Aluno A
 1 file changed, 1 insertion(+)
 create mode 100644 contribuicao-a.txt
henrique:~/roteiro_nivelamento_ci_expert$ git push
Username for 'https://github.com': henriquecunha2
Password for 'https://henriquecunha2@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 24 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 336 bytes | 336.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/henriquecunha2/roteiro_nivelamento_ci_expert
   fbe3841..0b7c1f0  main -> main
```

Git

- Parte 6, gerenciamento de conflitos
- Tentem as operações abaixo ao mesmo tempo, cada um em seu repositório! Quem fizer o *push* por último, terá que resolver o conflito!
- Ambos abrem o mesmo arquivo (ex.: README.md) e editam a mesma linha.
- Aluno X faz git add, commit e push.
- Aluno Y faz git add, commit e push → erro! O Git rejeita.
- Aluno Y faz git pull → conflito é detectado.
- Resolver o conflito manualmente no arquivo (escolher o que manter), depois:

Git

```
git add README.md
git commit -m "Resolve conflito no README"
git push
```

Não coloquei os prints aqui de propósito, para que vocês leiam o que está acontecendo e entendam se o conflito foi resolvid

Git

Parte 7 – branches

- Alterações são feitas em "galhos" diferentes do principal para evitar mexer no que está funcionando diretamente
- Antes de começar essa parte do roteiro, é recomendada a leitura da seguinte referência:
- https://git-scm.com/book/pt-br/v2/Branches-no-Git-Branches-em-poucas-palavras
- Se você entende como branches funcionam, pode pular
- Se está pareado com um colega que sabe sobre branches, peça a ele/ela uma explicação

Git

Parte 7 – branches

Aluno A cria um novo branch no nosso projeto, chamado Tcl_Hello

```
henrique:~/roteiro_nivelamento_ci_expert$ git branch Tcl_Hello
```

Git

Com esse comando, vemos que o HEAD está apontando para o *branch main*:

```
henrique:~/roteiro_nivelamento_ci_expert$ git log --oneline --decorate
0b7c1f0 (HEAD -> main, origin/main, Tcl_Hello) Adiciona arquivo do Aluno
A
fbe3841 Primeiro commit: adiciona README
```

Git

Observe a diferença quando mudamos para a branch Tcl_Hello e rodamos o mesmo comando:

```
henrique:~/roteiro_nivelamento_ci_expert$ git checkout Tcl_Hello
Switched to branch 'Tcl_Hello'
henrique:~/roteiro_nivelamento_ci_expert$ git log --oneline --decorate
0b7c1f0 (HEAD -> Tcl_Hello, origin/main, main) Adiciona arquivo do Aluno
A
fbe3841 Primeiro commit: adiciona README
henrique:~/roteiro_nivelamento_ci_expert$
```

Git

Aluno B não cria branch nenhuma, apenas faz uma alteração na branch main, em seguida, faz commit e push. Após a alteração, rode o seguinte comando:

```
git log --oneline --decorate --graph –all
```

O que é possível de observar de diferente?

Git

Parte 7 – merge e criação do primeiro arquivo .tcl

Aluno A cria um arquivo chamado hello.tcl, nele deve haver o seguinte conteúdo:

```
puts "Olá, Mundo - Entre aspas"    ;# Este é um comentário após o comando.

# Este é um comentário no início de uma linha

puts {Olá, Mundo - Entre chaves}

puts {Exemplo de sintaxe de comentário inválida}   # *Erro* - não há ponto e vírgula!

puts "Esta é a linha 1"; puts "esta é a linha 2"

puts "Olá, Mundo; - Com um ponto e vírgula dentro das aspas"

# Palavras não precisam ser colocadas entre aspas, a menos que contenham espaços:

puts OláMundo
```

Git

Se fizer com ctrl+c, ctrl+v pode dar erro.

Para rodar esse arquivo, basta digitar no terminal:

```
tclsh hello.tcl
```

Se funcionou, podemos seguir em frente com o merge

Git

Aluno A adiciona o arquivo no repositório, faz *commit* e *push* no *branch* anteriormente criado, chamada Tcl_Hello

Aluno B faz pull no repositório. Não deve nenhuma mudança na branch main, uma vez que as alterações de aluno A foram em outra branch

Aluno A agora faz merge nas *branches* da seguinte forma:

```
git checkout master
```

```
git merge Tcl_Hello
```

Git

Aluno B agora faz git pull novamente.

O que aconteceu?

Se o aluno B ainda não vê as alterações, o que será que é necessário para que elas possam aparecer?

Além disso, pesquise como apagar a branch que já não é mais necessária.

Git

Para mais conteúdo sobre branches e merge:

https://learngitbranching.js.org/

https://git-scm.com/docs/gittut

Git + tcl

Objetivos de Aprendizado

- Manipular arquivos de texto com TCL
- Usar expressões regulares para extrair informações estruturadas
- Processar dados de design digital (células, portas, conexões)
- Gerar relatórios automáticos úteis para fluxos de projeto
- Fazer tudo isso em dupla, usando Git (e Github) e mexendoem um único arquivo com branches e dividindo as tarefas

Git + tcl

Roteiro de tcl:

- Baixem o arquivo de netlist
- **Tarefa 1:**
- Leia o arquivo netlist.v
- Conte quantas instâncias de cada tipo de célula existem (AND2, XOR2, flipflop_D)
- Gere um relatório como:

```
=== RELATÓRIO DE CÉLULAS ===
AND2: 11 instâncias
XOR2: 9 instâncias
flipflop_D: 4 instâncias
TOTAL: 24 instâncias
```

Git + tcl

Tarefa 2:

- Identifique todos os módulos no arquivo
- Para cada módulo, liste quais submódulos ele instancia
- Gere uma árvore hierárquica

```
=== HIERARQUIA DO DESIGN ===
somador_4bits
  └─ (apenas células primitivas)

contador_4bits
  ├─ flipflop_D (4 instâncias)
  └─ (células primitivas)

flipflop_D
  └─ (módulo primitivo - sem submódulos)
```

Git + tcl

Tarefa 3:

- Gerar estatísticas de conexões
- Crie estatisticas_conexoes.tcl que:
- Identifique todas as nets (fios) no design
- Calcule o fanout de cada net (quantas entradas ela alimenta)
- Gere um relatório das 10 nets com maior fanout
- Identifique nets com fanout zero (possíveis erros)

```
=== TOP 10 NETS POR FANOUT ===
w9: fanout = 3
clk: fanout = 4
rst: fanout = 4
...

=== NETS COM FANOUT ZERO (POSSÍVEIS ERROS) ===
w_carry3
...
```

Git + tcl

Tarefa 4:

- Gerador de Relatório Completo
- Os alunos devem trabalhar paralelamente no mesmo arquivo, criando *branches,* fazendo merge e resolvendo conflitos
- Ao final do desenvolvimento, o gerador de relatório deve ser uma amalgama dessa colaboração
- Quando o repositório desse projeto estiver criado, enviem pelo seguinte form:
- Roteiro de Git+tcl – Preencher o formulário

Obrigado!

EXECUÇÃO:

iRede UnB UFRGS UNIFEI Universidade Federal do Ceará

APOIO: synopsys

COORDENAÇÃO: Softex

INICIATIVA: CHIP TECH BRASIL

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil